CLASE 10 · GUÍA DE TRABAJO

Entender, ejecutar y exportar tu modelo

Preparada para Griselda Blanco · Seminario de Actualización · IFTS N.º 18
Material de base: Cynthia M. Villagra · Revisión: 25/10/2026

1. Qué tenés que hacer

Vas a trabajar con una notebook: un documento que combina explicación, código y resultados. El ejercicio genera datos de 200 casas y aprende a estimar un precio a partir de tres características: metros cuadrados, balcón y ascensor. Son datos sintéticos; no es una aplicación de tasación real.

La tarea tiene tres partes: **recorrer el entrenamiento**, **escribir tus anticipaciones y conclusiones**, e **investigar un formato de exportación**. Elegiste modelo completo de PyTorch. La parte técnica ya tiene una copia ejecutada, pero falta tu trabajo de lectura, respuestas personales y revisión de la investigación.

2. Qué significa “faltan predicciones”

Dos usos de la palabra	Qué significa
Tu predicción personal	Una idea escrita antes de correr la siguiente celda: qué creés que va a pasar y por qué. No importa si después te equivocás.
La predicción del modelo	El número que calcula el programa para una casa. Por ejemplo, un precio estimado a partir de sus características.

No tenés que crear otro modelo ni buscar más datos. Debés completar las celdas de texto “Tu respuesta” y “Tu estimación”. Una respuesta puede empezar así: “Antes de ejecutar, creo que… porque…”. Después agregás: “Al ejecutarlo observé…; mi idea se confirmó / cambió porque…”.

Como ya hay una copia ejecutada, si viste una salida, aclaralo: “Esta respuesta es posterior a ver el resultado”. Podés repetir el recorrido como práctica, pero no presentarlo como una predicción previa que no hiciste. La revisión personal sigue siendo necesaria aunque el código funcione.

Trabajar en Google Colab

Camino recomendado: no necesitás instalar Python en tu PC. Los comandos de esta página se escriben en una **celda de código de Colab**, no en PowerShell.

Paso 1. Abrir el archivo

Entrá a colab.research.google.com. Usá Archivo > Subir notebook y seleccioná **Clase_10_Blanco_Griselda.ipynb** de tu carpeta Clase 10. Si preferís empezar con el material sin salidas, abrí el original **Clase_10_Notebook_Camino_Entrenamiento.ipynb**. Conservá el original y guardá tu trabajo con otro nombre.

Paso 2. Preparar las bibliotecas

En una celda nueva al principio, ejecutá esta instalación. Puede tardar unos minutos. Si Colab pide reiniciar la sesión, hacelo y volvé al principio.

```
%pip install -q numpy pandas matplotlib scikit-learn torch keras joblib
```

Luego, antes de cualquier importación de Keras:

```
import os
os.environ['KERAS_BACKEND'] = 'torch'
```

La copia preparada ya configura ese backend. Si usás el original, agregá las dos líneas antes de importar Keras. No necesitás GPU para este modelo pequeño. Keras y PyTorch siguen teniendo APIs diferentes aunque Keras use PyTorch como motor de cálculo.

Paso 3. Ejecutar con pausas

Usá el botón de reproducción de cada celda o **Shift + Enter**. Seguí el orden. Cuando aparezca PREDICCIÓN, detenete: abrí con doble clic la celda de texto siguiente, escribí tu idea y recién después ejecutá el código. Evitá “Ejecutar todas” durante este trabajo personal.

Paso 4. Guardar evidencia

Después de cada bloque, verificá que haya una salida, tabla o gráfico y escribí qué entendiste. Al terminar, descargá el documento con Archivo > Descargar > Descargar .ipynb. Ese archivo debe conservar las salidas y tus textos. Los archivos del entorno de Colab son temporales: descargá también los modelos que quieras conservar.

Nombres técnicos y bibliotecas

Nombre de la tarea: aprendizaje supervisado para regresión. **Tipo de modelo:** regresión lineal múltiple, porque usa tres entradas para estimar una salida numérica continua. No es un clasificador: no elige una categoría.

Implementación	Nombre y función
scikit-learn: LinearRegression	Estimador de regresión por mínimos cuadrados. Aprende coeficientes e intercepto; aquí no usa el ciclo de SGD.
scikit-learn: SGDRegressor	Otro estimador de regresión que ajusta parámetros mediante descenso de gradiente estocástico. En el ejercicio permite ver las correcciones iterativas.
PyTorch: nn.Linear(3, 1)	Capa lineal de tres entradas y una salida. Puede verse como una red de una sola capa lineal; MSELoss y SGD definen cómo se entrena.
Keras: Sequential + Dense(1)	Modelo secuencial con una capa de una salida y activación lineal predeterminada. compile configura el entrenamiento y fit lo ejecuta.

Biblioteca / importación	Para qué se usa
NumPy / numpy as np	Arreglos, generación de datos y operaciones numéricas.
pandas / pandas as pd	Tabla de casas y selección de columnas.
Matplotlib / matplotlib.pyplot as plt	Gráficos de datos y pérdida durante el entrenamiento.
scikit-learn / sklearn	Separar train/test, escalar, entrenar regresores y medir resultados. El nombre de instalación es scikit-learn.
PyTorch / torch	Tensores, gradientes automáticos, capa lineal y optimizador.
Keras / keras	API de alto nivel para construir, entrenar y guardar el modelo. En nuestra copia usa backend torch.
joblib	Guardar y cargar el pipeline de scikit-learn o el scaler. No entrena un modelo.
os, pathlib y json	Biblioteca estándar de Python: configuración, rutas y resultados JSON. No se instalan por separado.

Diferencia con Clase 7: allí se hace inferencia con modelos preentrenados de Hugging Face, como E5. En Clase 10 se entrenan regresores pequeños con el dataset del ejercicio. No necesitás un LLM, un token de Hugging Face, Firebase ni la app Expensa House para completar esta notebook.

El camino del entrenamiento

La fórmula que estás aprendiendo

```
precio_estimado = w1 * m2 + w2 * balcon + w3 * ascensor + b
```

Entradas o features: m2, balcon y ascensor. Balcón y ascensor valen 0 o 1.
Objetivo o target: precio, en miles de dólares.
Parámetros: w1, w2, w3 y b. El entrenamiento los ajusta.
Hiperparámetros: tasa de aprendizaje, épocas y tamaño de lote. Los configura quien entrena.

Paso 5. Entender qué hace cada parte

Parte	Qué mirar
Datos y separación	Se usan 160 casas para entrenamiento y 40 para test. Test permite evaluar con ejemplos que no se usaron para ajustar los parámetros.
Escalado	StandardScaler aprende medias y desvíos solo con train; luego transforma train, test y nuevas entradas con esos mismos valores. No escala el precio en este ejercicio.
Predicción y pérdida	El modelo calcula un precio. MSE promedia errores al cuadrado; la pérdida indica cuánto se alejó de los precios reales.
Gradiente y optimizador	backward calcula gradientes; step actualiza parámetros. Una época es un recorrido por el conjunto de entrenamiento.
Evaluación	RMSE se expresa en las unidades del precio. R^2 compara el ajuste con una referencia de precio medio. No son garantías de exactitud sobre casas reales.

Las líneas esenciales de PyTorch

```
y_pred = modelo_pt(Xt)
perdida = criterio(y_pred, yt)
optimizador.zero_grad()
perdida.backward()
optimizador.step()
```

Leelas como: **predice** → **mide el error** → **borra gradientes anteriores** → **calcula gradientes nuevos** → **corrige parámetros**. Repetir este bloque ajusta el modelo. zero_grad no borra los pesos; evita acumular gradientes de pasos anteriores.

Dónde escribir tus predicciones

Paso 6. Buscá cada título con Ctrl + F dentro de la notebook. Estas son las seis consignas; la primera incluye tres estimaciones. No hace falta acertar ni usar vocabulario perfecto.

Consigna	Qué hacés antes de ejecutar
PREDICCIÓN 1: estimar pesos	Mirás el gráfico y las tablas indicadas. Completás w1 con la subida aproximada por m², w2 con la diferencia asociada al balcón y w3 con la del ascensor. Escribí una estimación, no copies los coeficientes finales.
PREDICCIÓN 2: pesos al azar	Decís si esperás que la predicción inicial esté cerca o lejos del precio real y explicás tu razón. No necesitás calcular otro entrenamiento.
PREDICCIÓN 3: SGD y LinearRegression	Anticipás si los dos caminos llegarán a valores parecidos de w y b. Explicás en qué basás esa expectativa.
PREDICCIÓN 4: tasa 0,1 a 1,0	Anticipás si aumentar el tamaño de la corrección mejorará o empeorará el aprendizaje. Después comparás con las pérdidas impresas.
PREDICCIÓN 5: Keras	Estimás cuántas líneas visibles necesitará Keras para ejecutar el ciclo que PyTorch muestra como un for. Después distinguís el comando visible del trabajo interno.
PREDICCIÓN 6: modelo sin scaler	Anticipás qué podría pasar al cargar un regresor entrenado con entradas escaladas y darle datos originales. No hace falta adivinar un precio exacto; justificá el efecto esperado.

Plantilla para pegar en cada celda de texto

```
Antes de ejecutar, creo que:
[mi idea]

Lo pienso porque:
[mi razonamiento]

Despues de ejecutar observe:
[la salida real]

Comparacion con mi idea:
[que se confirmo o que cambio y por que]
```

Si ya habías visto la salida, agregá esa aclaración en vez de presentar la idea como anticipada. Si no entendés un resultado, también podés escribir qué parte te genera dudas. Esa comparación es justamente lo que busca la actividad.

Tu formato: modelo completo PyTorch

Paso 7. Exportar y recargar sin reentrenar

La sección de investigación ya está agregada al final de la copia preparada. Ejecutala después de entrenar modelo_pt y ajustar scaler. Este bloque usa variables creadas por el recorrido anterior; no funciona en un entorno vacío sin esos pasos.

```
from pathlib import Path
import torch
import joblib

carpeta = Path('modelos_exportados')
carpeta.mkdir(exist_ok=True)
torch.save(modelo_pt, carpeta / 'casas_completo.pt')
joblib.dump(scaler, carpeta / 'casas_completo_scaler.joblib')

# Cargar SOLO los archivos de confianza creados por vos.
recuperado = torch.load(
    carpeta / 'casas_completo.pt',
    map_location='cpu', weights_only=False
)
recuperado.eval()
scaler_guardado = joblib.load(
    carpeta / 'casas_completo_scaler.joblib'
)
```

torch.save serializa la instancia de nn.Linear y sus tensores. **No agrega automáticamente el scaler ni el estado del optimizador.** Por eso guardamos el preprocesamiento aparte. Para clases personalizadas puede seguir haciendo falta su definición importable; guardar el objeto no equivale a empaquetar todo el programa.

Paso 8. Comprobar la equivalencia

```
entrada = torch.tensor(
    scaler_guardado.transform(casa_nueva), dtype=torch.float32
)
with torch.inference_mode():
    antes = modelo_pt(entrada)
    despues = recuperado(entrada)
print('Antes:', antes.item())
print('Despues:', despues.item())
assert torch.allclose(antes, despues)
```

Acá no aparece fit ni un ciclo de entrenamiento: solo carga y predicción. **eval()** configura el modo de evaluación; **inference_mode()** evita registrar gradientes. weight y bias permiten inspeccionar los parámetros recuperados.

Seguridad: weights_only=False permite cargar objetos pickle; usalo solamente para tus archivos confiables. No ejecutes esta carga con modelos de desconocidos. Los archivos joblib requieren la misma precaución.

Qué explicar en la actividad

Paso 9. Revisar las cuatro preguntas con tus palabras

Pregunta	Puntos que tenés que poder explicar
¿Qué guarda?	La instancia del modelo y sus parámetros; ubicá w y b. Aclarar que el scaler va aparte y que no se guardó todo el estado de entrenamiento.
¿Qué necesito para cargar?	PyTorch y un entorno compatible. Con nn.Linear no reescribís la arquitectura para esta carga; una clase propia puede necesitar su módulo original.
¿Es seguro un archivo desconocido?	No por el hecho de ser .pt. La carga de objetos usa pickle y puede ejecutar código. Citar la documentación oficial; no probarlo con archivos desconocidos.
¿Para qué lo elegiría?	Ejemplo: una prueba Python/Gradio en un entorno controlado. Para continuar entrenamiento falta estado del optimizador; para Android hacen falta un formato y runtime adecuados, no basta copiar el .pt.

Los dos errores intencionales

SGD sin escalar: puede lanzar una excepción o devolver números muy grandes, según versión y configuración. En nuestra ejecución no hubo excepción; eso no demuestra que aprendió bien. Describí lo que efectivamente aparece y comparalo con el entrenamiento escalado.

Regresor sin scaler: el programa puede cargar y devolver un número equivocado sin mostrar error. Aprendió en una escala y recibió entradas en otra. Un archivo que se abre no garantiza una inferencia correcta.

Cómo reconocer los archivos

Archivo	Qué representa
casas.joblib	Pipeline de scikit-learn: transformación más regresor.
casas_pt.pt	state_dict: parámetros de PyTorch; no es lo mismo que el modelo completo.
casas_completo.pt	Objeto de modelo completo de PyTorch, el formato que elegiste investigar.
casas_completo_scaler.joblib	Transformación de entradas usada por ese modelo.
casas.keras	Modelo de Keras; su scaler también se conserva aparte.
casas_sin_scaler.joblib	Ejemplo del error intencional. No usar como modelo final de producción.

El nombre técnico “modelo entrenado” se refiere a la función con parámetros ya ajustados. El archivo exportado es su **artefacto serializado**. La biblioteca, la clase del modelo y el formato del archivo son tres cosas diferentes.

Alternativa local y entrega

Windows: comandos para trabajar sin Colab

Abrí PowerShell. Estos comandos crean un entorno propio en Clase 10 e inician Jupyter Notebook. No necesitás esta alternativa si elegís Colab. Usá el requirements.txt que ya quedó en esa carpeta.

```
Set-Location -LiteralPath 'C:\Users\grise\Proyectos'
$curso = 'Tercer Año\Seminario de Actualizacion\Seminario2026'
Set-Location -LiteralPath (Join-Path $curso 'Clase 10')
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt notebook
.\.venv\Scripts\python.exe -m notebook
```

La última línea abre Jupyter en el navegador. Abrí la notebook y seguí el mismo orden de la guía. .venv aísla las bibliotecas; pip instala dependencias; notebook inicia la interfaz. Para detener ese servidor local, volvé a PowerShell y presioná Ctrl + C. No compartas enlaces de sesión que incluyan tokens.

Paso 10. Lista final para subir al portfolio

1. Anotarte personalmente en la planilla: **modelo completo / PyTorch**.
2. Completar las seis predicciones, con las tres estimaciones de la primera.
3. Conservar salidas, gráficos, lectura de errores y comparación con tus ideas.
4. Revisar las cuatro respuestas de exportación y la prueba de recarga.
5. Descargar el .ipynb y los archivos exportados que vas a entregar.
6. En la rama **Proyecto_Integrador**, actualizar **clase_10/** con esa versión personal. La subida anterior ya incluye la base técnica: actualizá esos archivos, sin borrar otras carpetas.
7. Verificar que el README principal enlace a clase_10/ y abrir la notebook desde GitHub para comprobar que se ve.
8. No subir .venv, tokens, contraseñas, cuentas ni archivos privados de Expensa House.

El texto de la clase fija la entrega antes del viernes **30/10/2026**. La inscripción en la planilla y la entrega del aula no se realizaron automáticamente.

Fuentes y material de referencia

Consigna y notebook de Clase 10 proporcionados por la estudiante.
scikit-learn: LinearRegression · StandardScaler
PyTorch: guardar y cargar modelos · Keras: backend y configuración
Tu carpeta clase_10 en GitHub. Consulta: 25/10/2026.

Guía preparada con asistencia de IA para explicar el trabajo. No reemplaza tus respuestas ni la revisión docente.